

Table des matières

Cours 06 — Orchestration avec Airflow.....	1
Objectifs.....	2
1. Ce qu'un ordonnanceur fait, et ce qu'il ne fait pas.....	2
2. Le DAG, et ce que ses flèches ne disent pas	3
3. La planification, et l'intervalle de données.....	4
4. Le rattrapage.....	5
5. Rejouable : la fenêtre, le journal, et la reconstruction	6
6. Ce qui rate, ce qui tarde, ce qui attend.....	6
7. Ce qui se partage se réserve	7
8. Le point de couture.....	7
Points clés à retenir.....	8
Exercices.....	9

Cours 06 — Orchestration avec Airflow

Théorie · Module 6/12 — Formation Data Engineering · ~40 min · intermédiaire

Où vous êtes : source → **ingestion** → lac → entrepôt → restitution. Le Cours 05 a fait du pipeline un programme qu'on lance d'une commande. Ce cours-ci décide quand cette commande part, sur quelle tranche de temps elle travaille, ce qui se passe quand elle rate, et ce qui arrive quand on la met en route avec trois mois de retard.

En 30 secondes :

- Un ordonnanceur ne transforme rien. Il décide quoi lancer, quand, dans quel ordre, et il garde la trace de ce qui est passé. Le travail reste au pipeline du Cours 05.
- Un DAG est un graphe de tâches sans cycle. Les flèches disent l'ordre, pas le temps qui passe.
- Chaque exécution reçoit un intervalle de données, la tranche qu'elle doit traiter. Ce n'est pas l'heure à laquelle elle tourne, et confondre les deux est l'erreur la plus coûteuse du cours.
- Le rattrapage crée une exécution par intervalle manqué. Il n'a de sens que si les tâches savent se restreindre à leur tranche.

- Une tâche rejouable se relance sans dégât. Deux mécaniques l'assurent ici, et elles ne se remplacent pas : la fenêtre choisit quoi regarder, le journal des chargements décide quoi charger.
- Ce qui rate se réessaie, ce qui tarde se signale, et ce qui se partage se réserve. Trois problèmes distincts, trois réponses distinctes.

L'idée du cours. Jusqu'ici, c'est vous qui tapiez `python -m pipeline` tout, quand vous y pensiez, sur tout ce que le journal des chargements ignorait. Ça tient tant qu'une seule personne s'en occupe et que rien ne presse. Ce cours confie la commande à un ordonnanceur, et cela change deux choses. D'abord, le temps entre dans le pipeline : une exécution ne traite plus « ce qui reste », elle traite une tranche nommée. Ensuite, l'échec devient une situation normale, qu'on prévoit, plutôt qu'un accident qu'on découvre le lendemain.

Objectifs

- Dire ce qu'un ordonnanceur fait, ce qu'il ne fait pas, et nommer les services qui le composent.
- Lire un DAG : ses tâches, ses dépendances, et ce que ses flèches ne disent pas.
- Distinguer l'intervalle de données d'une exécution de l'heure à laquelle elle tourne.
- Expliquer le rattrapage, et ce qu'il exige des tâches pour donner un résultat juste.
- Reconnaître les deux mécaniques d'idempotence d'un pipeline ordonnancé, et ce que chacune couvre.
- Choisir entre une nouvelle tentative, une alerte de retard et une réservation de ressource.
- Situer le point de couture entre ce cours et la formation Analytics Engineering.

1. Ce qu'un ordonnanceur fait, et ce qu'il ne fait pas

Labo : Labo_06.md, étape 6.

Un ordonnanceur de flux de données répond à quatre questions, et à quatre seulement.

La question	Ce qu'il en fait
Quoi lancer	il lit des définitions de graphes, et connaît chaque tâche par son nom
Quand	une expression de planification, et la date à partir de laquelle elle vaut
Dans quel ordre	les dépendances déclarées entre tâches, et rien d'autre

La question	Ce qu'il en fait
Ce qui s'est passé	l'état de chaque exécution et de chaque tâche, ses journaux, ses durées

Ce qu'il ne fait pas mérite d'être dit aussi clairement. Il ne transforme aucune donnée. Il ne connaît ni vos fichiers, ni votre entrepôt, ni ce que veut dire « charger un mois ». Il lance une commande et regarde son code de sortie. C'est exactement pour cela que le Cours 05 s'est terminé sur une porte d'entrée unique avec un code de sortie honnête : sans elle, il n'y aurait rien à ordonnancer.

Analogie — le contremaître. Il ne visse rien lui-même. Il sait qui fait quoi, dans quel ordre, à quelle heure, et il note ce qui a été fait. Si vous lui donnez une consigne floue, il la transmet floue. Si l'ouvrier fait mal son travail, le contremaître ne le rattrape pas : il le consigne.

Airflow, l'ordonnanceur de ce cours, n'est pas un programme mais une petite équipe de services. Il est utile de les nommer tout de suite, parce que vous les verrez démarrer et parce que chacun peut tomber seul.

Le service	Ce qu'il fait
l'ordonnanceur	décide quoi lancer et quand ; avec l'exécuteur local, il lance aussi les tâches
le lecteur de DAG	relit le dossier des définitions et inscrit ce qu'il y trouve
le serveur d'API	l'interface web, et l'API que les tâches appellent pour dire où elles en sont
le déclencheur	tient les attentes sans occuper de place : délais, capteurs différables
la base de métadonnées	l'état de tout ce qui précède, dans un PostgreSQL à elle

Le lecteur de DAG est un service à part depuis Airflow 3, et c'est une bonne nouvelle : une définition qui ne compile pas ne peut plus faire tomber l'ordonnanceur avec elle. Vous verrez au labo ce que donne un fichier fautif, et où le dire.

À retenir — la base de métadonnées n'est pas votre entrepôt. Elle ne contient rien de Boréal : que des états, des dates et des journaux. Un ordonnanceur qui partagerait la base qu'il alimente tomberait avec elle, et ne pourrait même plus dire pourquoi.

2. Le DAG, et ce que ses flèches ne disent pas

Labo : Labo_06.md, étape 8.

DAG veut dire graphe acyclique dirigé : des tâches, des flèches entre elles, et aucun chemin qui revienne sur ses pas. La contrainte n'est pas décorative. Sans elle, il n'existerait pas d'ordre de passage, et l'ordonnanceur ne saurait pas par où commencer.

Le DAG quotidien de Boréal se lit d'une ligne : bronze → raw → corpus → dbt_build.

Tâche	Ce qu'elle lance
bronze	<code>python -m pipeline bronze</code> , qui refait le Parquet de l'usine
raw	<code>python -m pipeline raw</code> , limité à la journée de l'exécution
corpus	<code>python -m pipeline corpus</code> , qui dépose les documents
dbt_build	rien, pour l'instant : la place que la formation Analytics Engineering remplira

Trois choses que ces flèches ne disent pas, et qu'on croit souvent qu'elles disent.

Elles ne disent pas le temps. Une flèche veut dire « pas avant que », pas « juste après ». Entre la fin de `bronze` et le début de `raw`, il peut s'écouler une seconde ou une heure, selon ce que l'ordonnanceur a d'autre à faire.

Elles ne disent pas le passage de données. Rien ne circule le long d'une flèche. Si `raw` a besoin de ce que `bronze` a écrit, c'est parce que les deux touchent le même entrepôt, pas parce qu'une flèche les relie. Le graphe ordonne, il ne transporte pas.

Elles ne disent rien de ce qui est en dehors du graphe. Une tâche qui écrit dans un fichier qu'une autre tâche d'un autre DAG lit au même moment : le graphe ne le voit pas. C'est le problème de la section 7.

Piège — le DAG qui raconte l'ordre dans lequel on a écrit les tâches. La tentation est de tout enchaîner en ligne parce que c'est plus rassurant à lire. Une dépendance qui n'existe pas coûte du temps à chaque exécution, et masque les vraies. Ne mettez une flèche que là où la seconde tâche échouerait, ou mentirait, si la première n'avait pas fini.

3. La planification, et l'intervalle de données

Labo : Labo_06.md, étapes 1, 2, 8 et 9.

C'est la section qu'il faut lire deux fois. Elle contient la seule idée d'Airflow qui se comprend mal, et l'erreur qu'elle cause coûte des données fausses plutôt qu'une panne visible.

Une exécution planifiée porte deux temps, qui n'ont rien à voir l'un avec l'autre.

Le temps	Ce qu'il est
l'intervalle de données	la tranche que cette exécution doit traiter : du 15 mars au 15 mars
l'heure d'exécution	le moment où la machine a effectivement lancé la tâche : aujourd'hui, 16 h 23

Le premier est une propriété des données. Le second est un accident de calendrier. Une exécution du 15 mars rejouée aujourd'hui traite toujours le 15 mars, et c'est précisément ce qui la rend rejouable.

Une tâche écrit donc ses dates à partir de l'exécution, jamais à partir de l'horloge. Airflow fournit pour cela des valeurs à substituer dans une commande, et `{{ ds }}` est la plus utile : la date logique de l'exécution, en AAAA-MM-JJ. Pour une exécution planifiée, c'est le début de son intervalle. Pour une exécution lancée à la main sur le 15 mars, c'est le 15 mars, alors que l'intervalle qu'Airflow lui calcule part de l'heure du lancement : une commande lit donc la date logique, pas le début de l'intervalle.

```
python -m pipeline raw --depuis {{ ds }} --jusqua {{ ds }}
```

Le jour où vous rejouez le 15 mars, cette commande redemande le 15 mars. Si elle avait été écrite avec la date du jour, elle aurait rechargé aujourd'hui, et vous auriez cru avoir réparé le 15 mars.

Piège — l'horloge dans la commande. `date +%F` dans un `bash_command`, `datetime.now()` dans une fonction de tâche : les deux donnent l'heure de la machine, pas l'intervalle de l'exécution. Le pipeline tourne, ne signale rien, et remplit le mauvais jour. Le seul indice arrive des semaines plus tard, quand quelqu'un remarque un trou.

La planification elle-même est plus simple : un rythme, et une date de début. Le rythme découpe le temps en tranches, en journées pour `@daily`, en mois pour `@monthly`. La date de début dit à partir de quand ces tranches existent. Une date de fin, facultative, dit jusqu'où. Une exécution part quand sa tranche est finie : celle du 15 mars à minuit le 16, celle de janvier le 1er février.

Rappel — Airflow 3 a changé le sens de `@daily`. Écrivez seule, l'expression ne découpe plus de tranche : l'exécution de minuit porte la date du jour qui commence, et une commande qui lit `{{ ds }}` charge une journée encore vide, sans jamais échouer. Pour retrouver l'intervalle de données, le DAG l'écrit en toutes lettres : `schedule=CronDataIntervalTimetable("@daily", timezone="UTC")`. Un réglage de la plateforme fait la même chose pour tous les DAG, mais un DAG qui compte sur lui charge le mauvais jour le jour où on le dépose ailleurs. La documentation d'Airflow 2 suppose l'ancien sens.

Piège — la date de fin est inclusive, et l'exécution de trop est verte. Une fin posée au premier avril fait naître l'exécution du mois d'avril, en plus des trois qu'on voulait. On s'attendrait à un rouge : il n'y en a pas. Le chargeur ne trouve aucun fichier dans sa fenêtre, rend zéro ligne, et l'étape passe à « réussi » avec un code de sortie nul. Le DAG affiche donc quatre exécutions vertes, dont une qui n'a rien chargé. C'est le pire des résultats, et le motif que le Cours 11 reprendra : une exécution verte à zéro ligne ne se voit que si l'on a déclaré ce qu'on attendait. Posez la fin au dernier jour de la dernière tranche voulue.

4. Le rattrapage

Labo : Labo_06.md, étapes 5, 9 et 10.

Mettez en route un DAG mensuel dont la date de début remonte à trois mois, et l'ordonnanceur ne lancera pas une exécution : il en lancera trois, une par intervalle manqué, dans l'ordre. C'est le rattrapage, et c'est ce qui distingue un ordonnanceur d'une tâche planifiée du système d'exploitation, qui n'a aucune mémoire de ce qu'elle a manqué.

Le rattrapage n'a de valeur que sous une condition : l'état obtenu après les trois exécutions doit être celui qu'un chargement des trois mois d'un seul bloc aurait donné. Si les deux diffèrent, le rattrapage n'est pas une reprise, c'est une deuxième histoire.

Cette égalité ne va pas de soi. Elle exige que chaque tâche se limite à son intervalle. Un pipeline qui charge « tout ce qu’il ne connaît pas encore », comme celui du Cours 05, la casse d’une manière sournoise : la première exécution rattrapée avale les trois mois, les deux suivantes ne trouvent rien, et l’état final est juste. Juste, mais obtenu par hasard. Rejouez le deuxième mois seul, et vous n’obtiendrez rien du tout, alors que vous vouliez rejouer un mois.

D’où la fenêtre que le labo ajoute au pipeline : deux bornes, --depuis et --jusqua, qui disent à l’étape de chargement quels lots regarder.

À retenir — le rattrapage est une mesure, pas une promesse. Il se vérifie en chargeant deux fois les mêmes mois, une fois par exécutions successives, une fois d’un bloc, puis en comparant les tables ligne par ligne. C’est l’étape 6 du labo, et c’est la preuve de fin du cours.

Le rattrapage se refuse aussi, et parfois il le faut. Un DAG quotidien dont la date de début remonte à quatre ans créerait des milliers d’exécutions à la première mise en route. L’historique de Boréal est déjà chargé ; le DAG quotidien part donc sans rattrapage, et c’est le DAG mensuel qui rattrape.

5. Rejouable : la fenêtre, le journal, et la reconstruction

Labo : Labo_06.md, étapes 2, 3, 4 et 5.

Le Cours 03 a posé l’idempotence : relancer ne double rien. Le Cours 06 la pose une seconde fois, à l’échelle du DAG, et il faut voir que trois mécaniques distinctes y concourent, dans le seul pipeline de Boréal.

La mécanique	Ce qu’elle garantit	Où elle vit
la fenêtre	l’exécution ne regarde que les lots de sa tranche	--depuis et --jusqua
le journal des chargements	un fichier déjà chargé sous la même empreinte ne se recharge pas	pipeline.chargements, Cours 03
la reconstruction	réécrire l’ensemble redonne le même ensemble	le COPY ... OVERWRITE du Cours 02

Elles ne se remplacent pas, et le pipeline en emploie deux à la fois sans que ce soit une redondance. La fenêtre travaille sur des noms de fichiers : elle écarte ce qui n’est pas de la tranche sans rien lire. Le journal travaille sur des empreintes : il doit lire chaque fichier qui reste pour calculer son empreinte. Filtrer par la fenêtre d’abord, c’est ne pas relire 1 572 fichiers pour en charger 14.

Pourquoi — deux fenêtres qui se chevauchent ne doublent rien. C’est le journal qui l’assure, pas la fenêtre. Rejouez le mois de février après avoir chargé le trimestre, et le journal reconnaît chaque fichier : aucune ligne n’entre. La fenêtre décide du travail à examiner, le journal décide du travail à faire.

Toutes les étapes ne se fenêtrent pas, et c’est normal. Le bronze de l’usine réécrit ses partitions en entier à chaque passage, et son contrôle compare les totaux des deux côtés. Le dépôt du corpus compare des empreintes de documents, qui n’ont pas de date. Un DAG mélange donc des tâches incrémentales et des tâches qui reconstruisent tout, et savoir laquelle on écrit évite deux malheurs : recharger l’historique tous les matins, ou croire l’avoir rechargé.

6. Ce qui rate, ce qui tarde, ce qui attend

Labo : Labo_06.md, étape 8.

Trois situations se ressemblent de loin et appellent trois réponses différentes.

Une tâche rate. La nouvelle tentative s'en charge : un nombre d'essais et un délai entre deux. Elle vaut pour ce qui a une chance de passer au second coup, un verrou tenu par un autre processus, un service qui redémarre. Elle ne vaut rien contre une erreur de SQL, qui ratera aussi longtemps que personne ne l'aura corrigée, en occupant la place à chaque fois. La distinction est celle du Cours 05, entre erreur passagère et erreur définitive, remontée d'un cran.

Une tâche tarde. Elle n'a pas raté, elle n'a pas fini. C'est le cas le plus coûteux, parce que rien ne s'allume : un chargement qui prend trois heures au lieu de dix minutes finit peut-être par passer, et personne ne le sait. La réponse est une alerte de retard : un point de référence, un délai, et ce qu'on fait quand le délai est dépassé.

Rappel — les SLA d'Airflow 2 n'existent plus. Un SLA, un engagement de service, était un délai au-delà duquel une tâche en retard déclenchait une alerte. Airflow 3.0 les a retirés, callbacks et métriques compris, au profit des Deadline Alerts. Un DAG qui portait `sla=timedelta(...)` se migre à la main. Si vous cherchez « SLA airflow » dans un moteur de recherche, vous tomberez sur la documentation d'Airflow 2 et vous écrirez du code mort.

Une tâche attend. Un fichier qui n'est pas encore déposé, une table qu'un autre DAG doit remplir. Le capteur sert à cela : une tâche dont le travail est de vérifier une condition, régulièrement, jusqu'à ce qu'elle soit vraie. Un capteur différable ne garde pas sa place pendant qu'il attend : il la rend, et c'est le déclencheur qui surveille pour lui. Sur une petite installation, la différence se voit peu ; sur une grosse, dix capteurs bêtes bloquent dix places d'exécution pour ne rien faire.

7. Ce qui se partage se réserve

Labo : Labo_06.md, étapes 3 et 7.

Voici un problème que le graphe ne peut pas voir. Deux DAG différents, chacun correct, chacun avec ses propres dépendances, lancent au même moment une tâche qui écrit l'entrepôt DuckDB. Le second reçoit une erreur qui ne parle ni de DAG ni de tâche :

```
IO Error: Could not set lock on file "/projet/plateforme/entrepot/boreal.duckdb":  
Conflicting lock is held in /usr/python/bin/python3.13 (PID 94).
```

C'est la règle du Cours 03 qui remonte à la surface : un seul processus à la fois sur le fichier de l'entrepôt. Le nombre d'exécutions simultanées d'un DAG protège ce DAG de lui-même, et seulement de lui-même. Il faut donc un verrou que les deux DAG partagent.

Airflow appelle cela un pool : un nom, un nombre de places, et chaque tâche qui le déclare attend son tour. Un pool d'une seule place, réclamé par toute tâche qui appelle le pipeline, suffit à sérialiser les écritures sans changer une ligne du pipeline.

À retenir — un pool nomme une ressource, pas une tâche. Ce qu'on protège ici n'est pas « le chargement mensuel », c'est le fichier `boreal.duckdb`. Tout ce qui l'écrit prend le pool, quel que soit le DAG où il vit, et un lecteur du DAG comprend pourquoi en lisant le nom.

8. Le point de couture

Le DAG quotidien finit sur une tâche qui ne fait rien, nommée `dbt_build`. Ce n'est pas un oubli.

La chaîne de Boréal ne s'arrête pas à la RAW. Ce que le Cours 03 y a déposé sans le transformer, la formation Analytics Engineering le décode, le met en forme et le livre en tables de faits et de dimensions. Son outil s'appelle `dbt`, et il se lance par une commande, `dbt build`, exactement comme le pipeline se lance par `python -m pipeline`.

Cette tâche vide est donc l'endroit, nommé et daté, où les deux formations se rejoignent. La poser maintenant coûte une ligne. Ne pas la poser coûterait la refonte du DAG le jour où AE arrive, et personne ne saurait plus si `raw` doit précéder `dbt build` ou l'inverse.

Recoupements — ce que ce cours partage avec ses voisines. - Analytics Engineering. `dbt build` dans un DAG est le point de couture entre les deux formations : ce cours pose la tâche, AE la remplit et possède tout ce qui se passe après la RAW. - Machine Learning Engineering. Un DAG d'entraînement réemploie le même outil et les mêmes notions, intervalle de données compris ; ce qui change est ce que les tâches font.

Points clés à retenir

- Un ordonnanceur décide quoi lancer, quand et dans quel ordre, et garde la trace. Il ne transforme rien : le travail reste au pipeline, derrière une porte d'entrée unique.
- Une flèche d'un DAG dit « pas avant que ». Elle ne dit ni le délai, ni le passage de données, et elle ne voit rien de ce qui se passe hors du graphe.
- L'intervalle de données d'une exécution n'est pas l'heure à laquelle elle tourne. Une commande qui lit l'horloge remplit le mauvais jour sans jamais échouer. Depuis Airflow 3, le DAG écrit son intervalle en toutes lettres, et la commande lit la date logique.
- Le rattrapage crée une exécution par intervalle manqué. Il ne vaut que si chaque tâche se limite à son intervalle, et cela se vérifie en comparant avec un chargement d'un bloc.
- La fenêtre choisit quoi regarder, le journal des chargements décide quoi charger, la reconstruction réécrit tout. Trois mécaniques, trois rôles, et un DAG mélange les trois.
- Une tâche qui rate se réessaie, une tâche qui tarde se signale, une tâche qui attend se diffère. Les SLA d'Airflow 2 sont devenus les Deadline Alerts d'Airflow 3.
- Une ressource partagée entre DAG se réserve par un pool, qui nomme la ressource et non la tâche. Le nombre d'exécutions simultanées ne protège un DAG que de lui-même.
- Une tâche vide et nommée vaut mieux qu'un DAG à refaire : `dbt_build` est le point de couture avec Analytics Engineering.

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_06.md`. Faites-les après le labo : plusieurs énoncés supposent que les deux DAG existent, qu'ils ont tourné au moins une fois, et que la comparaison du rattrapage a été faite.